
INFORME TÉCNICO DE IMPLEMENTACIÓN

Implementación de Persistencia, CRUD, Automatización de Pruebas y Mejora de Clientes

Arquitectura Limpia con Java 21, Spring Boot y JavaFX

Autor: Juan Manuel Rengifo Fretel

Curso: Construcción de Software II

Docente: Ríos Rivera, Carlos Abraham

Semestre: 2026-II

26 de septiembre de 2026

Resumen

Este informe documenta la evolución de CacaoSelva, una prueba de concepto académica orientada a la gestión de lotes de cacao mediante una API REST, una aplicación de escritorio y un monitor automático. La versión inicial almacenaba tres registros en memoria y ofrecía únicamente operaciones de consulta. La solución final incorpora persistencia con JPA, una base H2 en archivo, un perfil alternativo para PostgreSQL, diez registros iniciales, operaciones CRUD, validaciones, una interfaz JavaFX mejorada, un monitor configurable y automatización de pruebas con GitHub Actions. La implementación conserva la dirección de dependencias propuesta por Arquitectura Limpia: dominio y aplicación permanecen independientes de Spring, JavaFX, HTTP y JPA. La verificación final ejecutó 21 pruebas sin fallos, compiló los siete proyectos del reactor Maven y confirmó la persistencia mediante la creación de un registro, el reinicio completo de la API y su posterior recuperación. Los resultados muestran que el sistema satisface los requisitos funcionales y técnicos definidos para la entrega, con una estructura preparada para evolución posterior.

Palabras clave: arquitectura limpia, persistencia, JavaFX, Spring Boot, JPA, pruebas automatizadas.

Índice

1	Introducción	4
2	Objetivos	4
2.1	Objetivo general	4
2.2	Objetivos específicos	4
3	Alcance y requisitos atendidos	5
4	Arquitectura de la solución	6
4.1	Responsabilidad de los módulos	6
5	Persistencia de datos	7
5.1	Decisión tecnológica	7
5.2	Modelo persistente	7
5.3	Carga inicial y conservación	7
6	API REST y métodos implementados	8
7	Análisis y mejora del cliente Desktop	8
7.1	Estado inicial	8
7.2	Estado mejorado	9
8	Análisis y mejora del Monitor	9
9	Estrategia de pruebas y automatización	11
9.1	Integración continua	12
10	Resultados de verificación	12
11	Guía de ejecución	13
11.1	Compilación y pruebas	13
11.2	API	13
11.3	Desktop y Monitor	13
11.4	Perfil PostgreSQL	13
12	Principios de diseño aplicados	14
13	Limitaciones y trabajo futuro	14
14	Conclusiones	14
	Referencias	16

1. Introducción

CacaoSelva demuestra cómo diferentes clientes pueden compartir información de negocio a través de una API HTTP sin acoplarse a los detalles de almacenamiento. El sistema se desarrolló como un proyecto Maven multimódulo en Java 21 y está compuesto por dominio, aplicación, infraestructura, API REST, Desktop JavaFX y Monitor. Esta división permite evaluar de forma práctica principios de diseño como responsabilidad única, inversión de dependencias y segregación de interfaces.

La versión de partida cumplía el objetivo de consulta, pero presentaba limitaciones importantes: los datos desaparecían al finalizar el proceso, el repositorio era de solo lectura, Desktop no permitía administrar lotes y el Monitor repetía el mismo mensaje en cada intervalo. Además, la automatización de pruebas todavía no cubría la persistencia ni los flujos completos de escritura.

El trabajo realizado aborda dichas limitaciones con cambios incrementales y verificables. Después de cada fase se ejecutó la suite completa para detectar regresiones antes de continuar. Esta estrategia redujo el riesgo de afectar los clientes existentes mientras se incorporaban nuevas capacidades.

2. Objetivos

2.1. Objetivo general

Evolucionar CacaoSelva hacia una solución persistente, verificable y mantenible, respetando su Arquitectura Limpia y garantizando que API, Desktop y Monitor continúen trabajando de forma integrada.

2.2. Objetivos específicos

1. Implementar almacenamiento persistente sin introducir dependencias de infraestructura en dominio o aplicación.
2. Alimentar la base con un conjunto inicial de datos representativo.

3. Completar las operaciones de creación, consulta, actualización y eliminación de lotes.
4. Automatizar pruebas unitarias y de integración tanto localmente como en GitHub.
5. Mejorar la experiencia y capacidad funcional del cliente Desktop.
6. Reducir ruido operativo y mejorar la recuperación del Monitor.
7. Documentar la conexión, las decisiones arquitectónicas y los resultados obtenidos.

3. Alcance y requisitos atendidos

La Tabla 1 resume la trazabilidad entre lo solicitado y lo implementado.

Cuadro 1. *Trazabilidad de requisitos*

Requisito	Implementación	Evidencia
Conexión persistente	H2 en modo archivo y perfil PostgreSQL	Reinicio real con recuperación del registro
Alimentación de datos	Carga condicional de 10 lotes	<code>InitialDataConfig</code>
Métodos pendientes	CRUD REST completo	GET, POST, PUT y DELETE
Automatización	Maven, JUnit y GitHub Actions	21 pruebas sin fallos
Diagrama simple	Diagrama de componentes y conexión	Figura 1
Mejora Desktop	Filtros, formulario y CRUD asíncrono	Figura 4
Mejora Monitor	Métricas, cambios, recuperación y cierre	Pruebas de estado y configuración
Publicación	Repositorio Git en rama <code>main</code>	Flujo CI en cada push y pull request

4. Arquitectura de la solución

La solución sigue una organización inspirada en Arquitectura Limpia. Martin (2018) sostiene que las políticas de negocio deben permanecer independientes de frameworks y mecanismos externos. En CacaoSelva esta condición se materializa manteniendo los módulos `domain` y `application` libres de anotaciones de Spring, clases de JavaFX, clientes HTTP y entidades JPA.

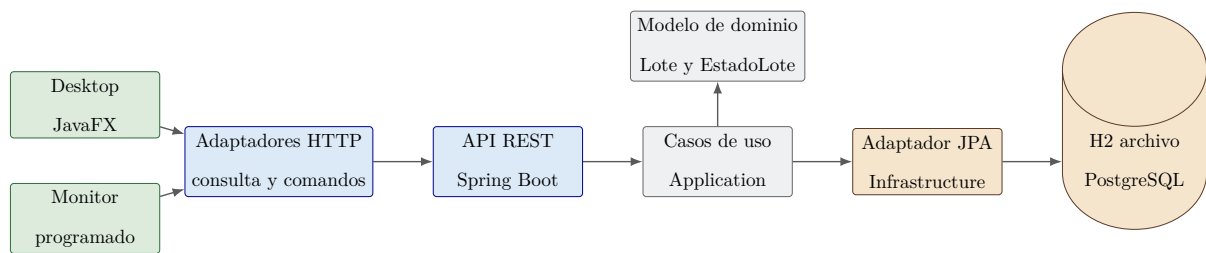


Figura 1. *Arquitectura y conexión principal de CacaoSelva.*

4.1. Responsabilidad de los módulos

Cuadro 2. *Responsabilidades por módulo*

Módulo	Responsabilidad
<code>domain</code>	Modelo puro: <code>Lote</code> y <code>EstadoLote</code> . No depende de frameworks.
<code>application</code>	Puertos, DTO, validaciones, excepciones y casos de uso. Define qué necesita el negocio, no cómo se implementa.
<code>infrastructure</code>	Implementaciones JPA y HTTP. Traduce entre mecanismos externos y contratos de aplicación.
<code>api</code>	Controladores REST, DTO de entrada y salida, configuración de Spring y manejo de errores.
<code>desktop</code>	Interfaz JavaFX para consulta y administración de lotes.
<code>monitor</code>	Consulta periódica, resumen de estados y detección de disponibilidad.

5. Persistencia de datos

5.1. Decisión tecnológica

La base predeterminada es H2 en modo archivo. H2 admite bases en memoria y persistentes en disco, además de operación embebida mediante JDBC (H2 Group, s. f.). Esta elección evita exigir una instalación adicional durante la demostración y permite comprobar persistencia real entre reinicios. Para un entorno compartido se añadió el perfil `postgres`, cuyas credenciales pueden suministrarse mediante variables de entorno.

Spring Data JPA reduce código repetitivo de acceso a datos mediante abstracciones de repositorio (Spring, s. f.). El adaptador `JpaLoteRepositoryAdapter` implementa el puerto definido en aplicación; de este modo, los casos de uso no conocen Hibernate, H2 ni PostgreSQL.

5.2. Modelo persistente

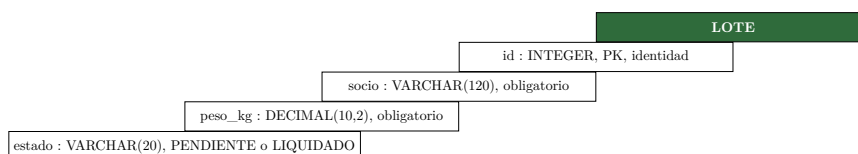


Figura 2. Estructura lógica persistida para un lote.

Las validaciones impiden socio vacío o mayor de 120 caracteres, peso nulo, no positivo, con más de dos decimales o fuera de la precisión definida, y estado ausente. Estas reglas se ejecutan en aplicación antes de invocar el repositorio, por lo que no dependen del canal REST.

5.3. Carga inicial y conservación

Al iniciar la API, `InitialDataConfig` consulta la cantidad de registros. Solo cuando la tabla está vacía inserta diez lotes con diferentes socios, pesos y estados. Esta condición evita duplicar o sobrescribir información en reinicios posteriores.

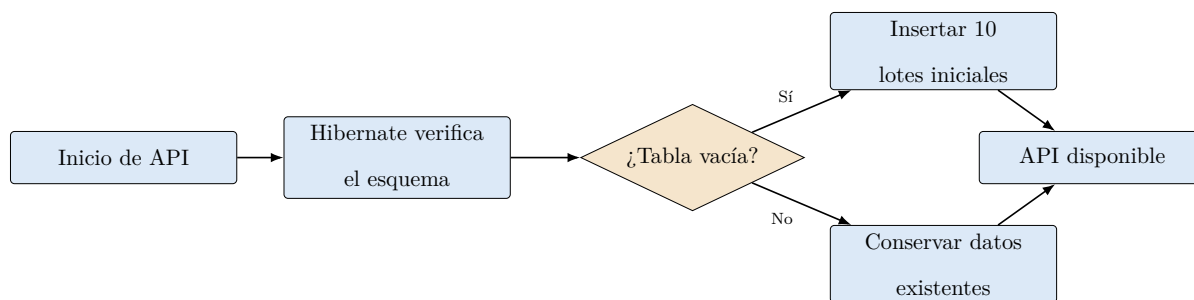


Figura 3. *Secuencia de inicialización persistente.*

6. API REST y métodos implementados

La API preservó los métodos de consulta y añadió las operaciones pendientes. Los controladores se limitan a traducir HTTP y delegan la lógica a los casos de uso.

Cuadro 3. *Contrato REST final*

Método	Ruta	Propósito	Respuesta
GET	/lotes	Listar registros ordenados por identificador	200
GET	/lotes/{id}	Buscar un lote	200, 400 o 404
POST	/lotes	Crear y asignar identificador	201
PUT	/lotes/{id}	Actualizar un lote existente	200, 400 o 404
DELETE	/lotes/{id}	Eliminar después de verificar existencia	204, 400 o 404

Los errores se representan con código, mensaje y marca temporal. Esta respuesta consistente facilita el manejo tanto en Postman como en Desktop.

7. Análisis y mejora del cliente Desktop

7.1. Estado inicial

La primera interfaz contenía un botón de consulta y una tabla de solo lectura. La llamada HTTP ya era asíncrona, lo que evitaba congelar JavaFX, pero el usuario no podía

alimentar ni corregir información. Tampoco existían filtros para trabajar cómodamente con listas mayores.

7.2. Estado mejorado

La versión final carga automáticamente los datos, mantiene el botón de actualización y agrega filtros por nombre de socio y estado. Un formulario permite crear nuevos lotes o editar el registro seleccionado. La eliminación exige confirmación y todos los comandos se ejecutan fuera del hilo gráfico mediante `CompletableFuture`.

Para conservar segregación de interfaces se creó `LoteCommandPort`, separado de `LoteQueryPort`. El controlador JavaFX depende de ambos contratos. El adaptador HTTP de comandos encapsula URI, JSON, códigos HTTP y tiempos de espera.

Cuadro 4. *Comparación del cliente Desktop*

Antes	Después
Consulta manual	Carga inicial y actualización manual
Tabla de solo lectura	Alta, edición y eliminación
Sin filtros	Filtros por socio y estado
Tres registros fijos	Datos persistentes y dinámicos
Mensaje genérico de conexión	Estados de carga, éxito y error
Sin confirmación de acciones	Confirmación previa a eliminar

La Figura 4 muestra la ejecución real del cliente Desktop conectado a la API. La tabla contiene los diez registros iniciales persistidos y presenta, en una sola vista, búsqueda, filtro de estado y formulario de gestión.

8. Análisis y mejora del Monitor

El Monitor inicial registraba únicamente el número de pendientes cada diez segundos. Esto generaba líneas repetidas aun cuando el sistema no cambiaba. La nueva versión construye un resumen con total, pendientes y liquidados, y solo lo registra cuando difiere del anterior.

Figura 4. *Cliente Desktop de CacaoSelva en funcionamiento*

CacaoSelva Gestion de lotes Actualizar

Buscar socio Estado TODOS

ID	Socio	Peso (kg)	Estado
1	Ana Torres	120.50	PENDIENTE
2	Luis Mendoza	80.00	LIQUIDADO
3	Rosa Quispe	95.25	PENDIENTE
4	Carlos Rojas	143.75	PENDIENTE
5	Mariela Sanchez	67.40	LIQUIDADO
6	Jose Paredes	110.20	PENDIENTE
7	Elena Huaman	89.60	LIQUIDADO
8	Miguel Salazar	156.00	PENDIENTE
9	Patricia Flores	72.35	PENDIENTE
10	Victor Castillo	101.90	LIQUIDADO

Datos del lote

Socio Peso

Estado PENDIENTE

Nuevo lote Nuevo Eliminar Guardar

Nota. La captura evidencia la consulta de datos persistidos, los filtros por socio y estado, y las acciones de creación, edición y eliminación disponibles en JavaFX.

Además, una caída de la API produce una única advertencia; los ciclos fallidos siguientes permanecen silenciosos hasta que la conexión se recupera. La recuperación se informa explícitamente. El intervalo acepta la variable `CACAOSELVA_MONITOR_INTERVAL_SECONDS` o la propiedad Java `cacaoselva.monitor.interval.seconds`. Un *shutdown hook* detiene el planificador y espera hasta cinco segundos antes de forzar el cierre.

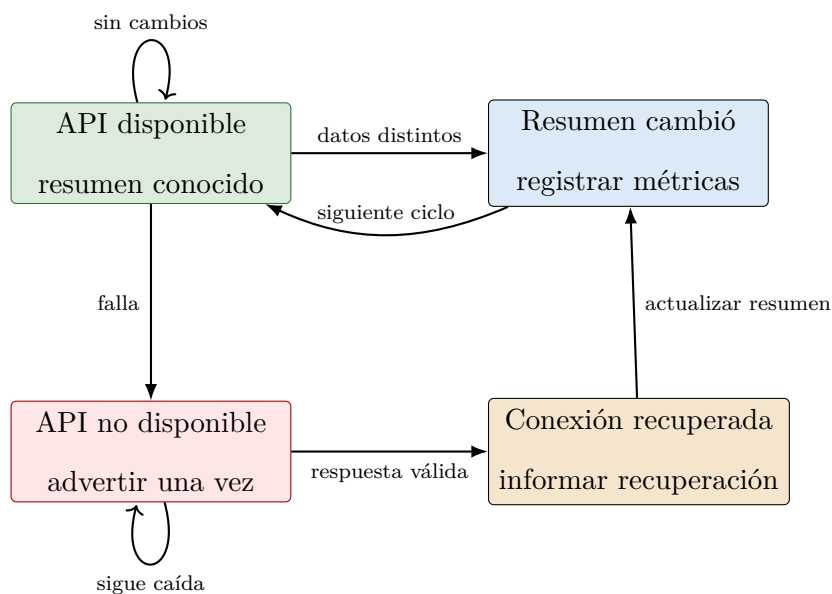


Figura 5. Estados operativos simplificados del Monitor.

9. Estrategia de pruebas y automatización

La suite combina pruebas unitarias rápidas con pruebas de integración. Los casos de uso emplean Mockito para verificar colaboración con puertos. La API levanta Spring con H2 en memoria y ejecuta el CRUD completo mediante `MockMvc`. El adaptador de comandos HTTP se prueba contra un servidor local controlado. El Monitor se prueba con puertos falsos y capturadores de mensajes, sin iniciar tareas infinitas.

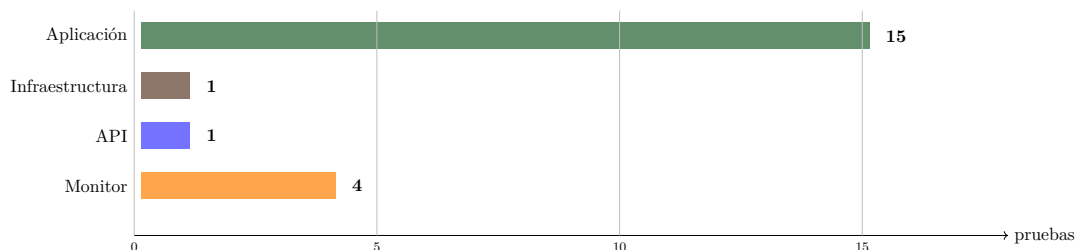


Figura 6. Distribución de las 21 pruebas automatizadas.

9.1. Integración continua

El archivo `.github/workflows/ci.yml` activa un flujo ante *push* y *pull request*. El trabajo descarga el código, configura Temurin Java 21, habilita caché de Maven y ejecuta `mvn -batch-mode -no-transfer-progress verify`. GitHub recomienda este patrón para compilar y probar proyectos Java con Maven en integración continua (GitHub, s. f.).

10. Resultados de verificación

La verificación final se ejecutó con `mvn clean verify`. Los siete elementos del reactor finalizaron correctamente: proyecto raíz, dominio, aplicación, infraestructura, API, Desktop y Monitor. El resultado agregado fue de 21 pruebas, cero fallos, cero errores y cero pruebas omitidas.

La persistencia también se comprobó fuera del entorno de pruebas:

1. Se inició el JAR empaquetado y se verificó su conexión persistente con H2 en modo archivo.
2. Se creó por HTTP un lote con identificador 11.
3. Se detuvo la API mediante cierre ordenado.
4. Se inició nuevamente el mismo JAR.
5. `GET /lotes/11` devolvió el registro creado antes del reinicio.
6. Se eliminó el registro técnico de verificación para conservar los diez datos iniciales previstos.

La aplicación Desktop fue ejecutada con la API activa y mostró los diez registros persistidos, el formulario, los filtros y las acciones de gestión. Esta observación confirma la ruta completa Desktop–HTTP–API–JPA–H2.

11. Guía de ejecución

11.1. Compilación y pruebas

Desde la raíz del proyecto:

```
mvn clean verify
```

11.2. API

```
java -jar api/target/api-1.0.0-SNAPSHOT.jar
```

La API queda disponible en <http://localhost:5080>. El archivo persistente se crea en `data/cacaoselva.mv.db`.

11.3. Desktop y Monitor

En terminales separadas, con la API activa:

```
mvn -pl desktop javafx:run  
mvn -pl monitor exec:java
```

Para cambiar el intervalo del Monitor en PowerShell:

```
$env:CACAOSELVA_MONITOR_INTERVAL_SECONDS = "30"  
mvn -pl monitor exec:java
```

11.4. Perfil PostgreSQL

Después de crear la base `cacaoselvabd`, se puede activar el perfil con:

```
mvn -pl api spring-boot:run -Dspring-boot.run.profiles=postgres
```

Las variables `DB_URL`, `DB_USERNAME` y `DB_PASSWORD` permiten evitar credenciales fijas.

12. Principios de diseño aplicados

- **SRP:** controladores, casos de uso, adaptadores, persistencia, UI y planificación tienen motivos de cambio distintos.
- **OCP:** H2 y PostgreSQL utilizan el mismo adaptador JPA; otro almacenamiento puede implementar el puerto sin modificar casos de uso.
- **LSP:** el repositorio JPA y el repositorio en memoria son sustituibles desde el contrato de aplicación.
- **ISP:** consulta y comandos HTTP se expresan en puertos separados.
- **DIP:** la lógica depende de `LoteRepository`, `LoteQueryPort` y `LoteCommandPort`, no de implementaciones concretas.

13. Limitaciones y trabajo futuro

La entrega prioriza arquitectura, persistencia y verificabilidad. No incluye autenticación, autorización, cifrado de secretos, migraciones versionadas ni despliegue productivo. H2 es adecuado para demostración local, pero PostgreSQL debe preferirse cuando varios procesos o usuarios compartan datos. Como evolución futura se recomienda incorporar Flyway, perfiles diferenciados por ambiente, pruebas visuales automatizadas de JavaFX, paginación, auditoría de cambios y autenticación basada en roles.

14. Conclusiones

CacaoSelva pasó de ser una consulta de datos volátiles a una solución persistente y administrable. La incorporación de JPA se realizó como un adaptador externo, conservando la independencia del núcleo. El CRUD amplió la utilidad de la API sin romper los contratos de consulta usados por Desktop y Monitor.

Las mejoras de Desktop proporcionan un flujo completo para el usuario, mientras que el Monitor ofrece información más útil con menos ruido. La automatización de 21

pruebas y el flujo de GitHub Actions reducen el riesgo de regresiones y hacen reproducible la verificación del proyecto.

Finalmente, la prueba de reinicio confirmó que la persistencia no es únicamente una configuración declarativa: los datos sobreviven al ciclo de vida de la API. En conjunto, la solución cumple los requisitos académicos planteados y mantiene una base técnica clara para extensiones posteriores.

Referencias

Apache Maven Project. (s. f.). *Maven in five minutes*. <https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html>

GitHub. (s. f.). *Building and testing Java with Maven*. GitHub Docs. <https://docs.github.com/>

H2 Group. (s. f.). *Features*. H2 Database Engine. <https://h2database.com/>

JUnit Team. (s. f.). *JUnit 5 user guide*. <https://junit.org/junit5/docs/current/user-guide/>

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Pearson.

Oracle. (s. f.). *JavaFX documentation*. <https://docs.oracle.com/>

Spring. (s. f.). *Spring Data JPA reference documentation*. <https://docs.spring.io/spring-data/jpa/reference/>